

Hexagonale Architektur in Python

Ein kompakter Ansatz für Ports & Adapters mit Domain, Application Layer, Infrastructure-Adaptern und Dependency Injection.

Grundidee

Die hexagonale Architektur (Ports & Adapters) eignet sich gut für Python-Projekte, wenn fachliche Logik von technischen Details wie Datenbanken, Web-Frameworks, Messaging oder externen APIs getrennt werden soll. Der Anwendungskern kennt die Außenwelt nicht. FastAPI, SQLAlchemy, PostgreSQL oder externe Services werden als Adapter angebunden.

Mögliche Projektstruktur

```
src/
|-- domain/
|   |-- entities.py
|   |-- value_objects.py
|   `-- services.py
|
|-- application/
|   |-- ports/
|   |   |-- user_repository.py
|   |   `-- notification_service.py
|   `-- use_cases/
|       `-- register_user.py
|
|-- adapters/
|   |-- inbound/
|   |   |-- api/
|   |   |   `-- fastapi_routes.py
|   |   `-- cli.py
|   `-- outbound/
|       |-- persistence/
|       |   |-- sqlalchemy_user_repository.py
|       `-- notifications/
|           `-- email_notification_service.py
|
`-- bootstrap.py
```

1. Port definieren

In Python bietet sich **typing.Protocol** für Ports an. Damit bleibt die Kopplung gering und Implementierungen müssen nicht zwingend von abstrakten Basisklassen erben.

```
from typing import Protocol
from domain.entities import User

class UserRepository(Protocol):
    def save(self, user: User) -> None:
        ...

    def find_by_email(self, email: str) -> User | None:
        ...
```

2. Use Case im Application Layer

Der Use Case kennt nur den Port und keine konkrete Datenbankimplementierung.

```
from application.ports.user_repository import UserRepository
from domain.entities import User

class RegisterUser:
    def __init__(self, users: UserRepository):
        self.users = users
```

```
def execute(self, email: str) -> User:
    if self.users.find_by_email(email):
        raise ValueError("User already exists")

    user = User(email=email)
    self.users.save(user)
    return user
```

3. Outbound Adapter für die Datenbank

```
class SqlAlchemyUserRepository:
    def __init__(self, session):
        self.session = session

    def save(self, user):
        self.session.add(user)
        self.session.commit()

    def find_by_email(self, email):
        return (
            self.session.query(User)
                .filter_by(email=email)
                .first()
        )
```

4. Inbound Adapter, z. B. FastAPI

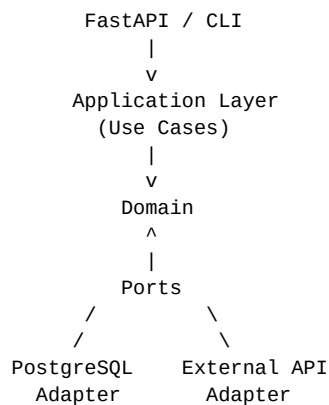
FastAPI ist nur ein Eintrittspunkt in die Anwendung. Die fachliche Logik bleibt im Use Case und ist dadurch unabhängig vom Web-Framework.

```
from fastapi import APIRouter

router = APIRouter()

@router.post("/users")
def register_user(request: RegisterUserRequest):
    use_case = get_register_user_use_case()
    return use_case.execute(email=request.email)
```

Abhängigkeitsrichtung



5. Dependency Injection / Composition Root

Für viele Python-Projekte ist kein Dependency-Injection-Framework erforderlich. Ein zentraler Composition Root reicht aus, um konkrete Adapter mit den Use Cases zu verbinden.

```
def create_application():
    session = create_db_session()
    repository = SQLAlchemyUserRepository(session)

    register_user = RegisterUser(
        users=repository
    )

    return Application(
        register_user=register_user
    )
```

6. Einfache Tests mit In-Memory-Adaptern

```
class InMemoryUserRepository:
    def __init__(self):
        self.users = {}

    def save(self, user):
        self.users[user.email] = user

    def find_by_email(self, email):
        return self.users.get(email)

def test_register_user():
    repository = InMemoryUserRepository()
    use_case = RegisterUser(repository)

    user = use_case.execute("alice@example.com")

    assert user.email == "alice@example.com"
```

Warum dieser Ansatz für Python sinnvoll ist

Testbarkeit: Fachliche Tests benötigen weder eine echte Datenbank noch FastAPI oder Docker.

Austauschbarkeit: Datenbank, API-Client oder UI können ersetzt werden, ohne den Anwendungskern umzubauen.

Klare Verantwortlichkeiten: Domain, Use Cases und Infrastruktur bleiben voneinander getrennt.

Python-typisch: Protocol und Duck Typing ermöglichen schlanke Ports ohne unnötige Vererbung.

Pragmatischer Start

Für kleinere Projekte sollte die Architektur nicht unnötig aufgebläht werden. Eine Struktur mit **domain/**, **application/**, **adapters/** und **main.py** ist häufig ausreichend. Weitere Unterteilungen nach Features oder Bounded Contexts können später ergänzt werden, wenn die Anwendung wächst.